IoT-integrated Smart Vending Machine



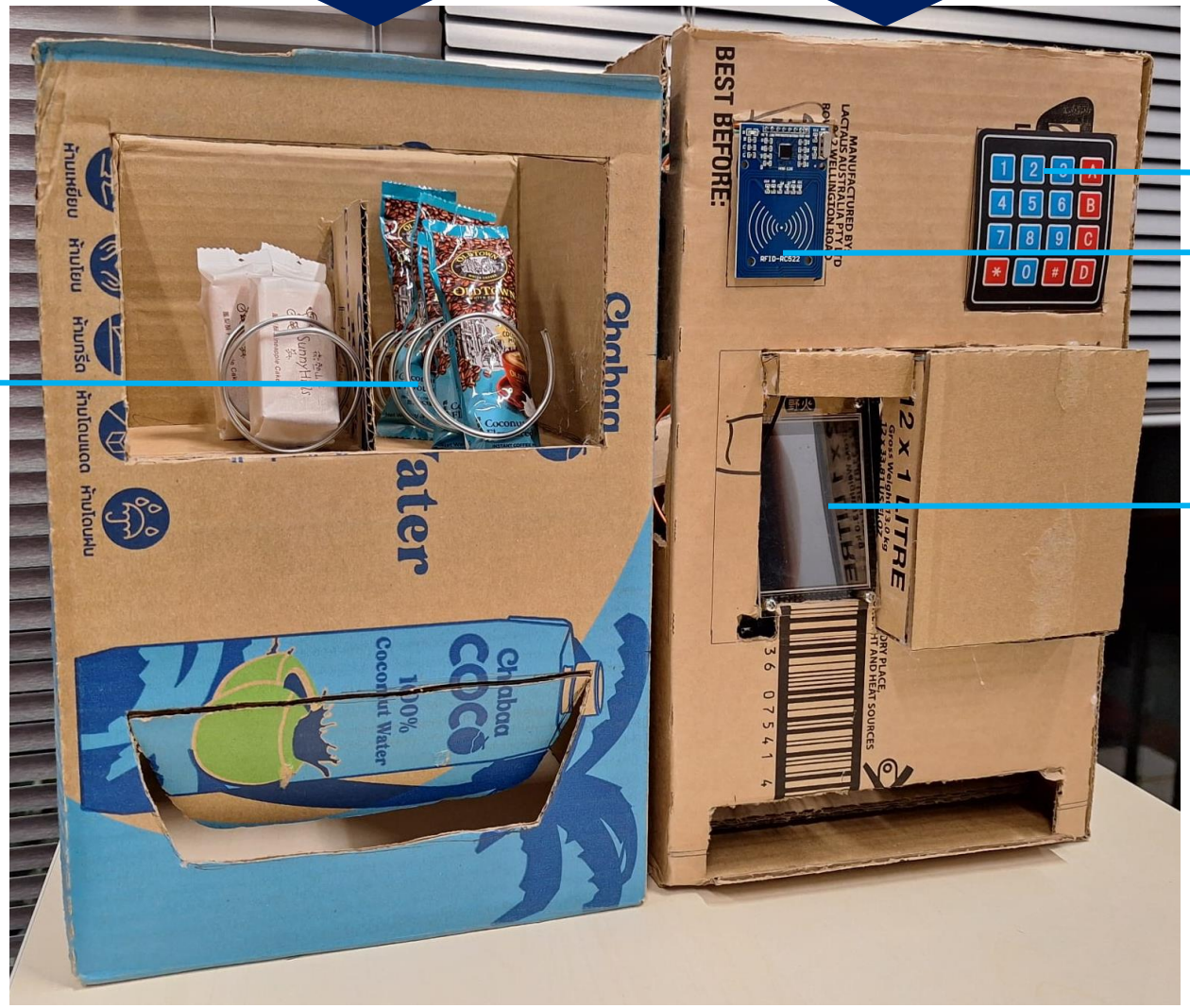
Group 1

Sawini, Samantha

Project Design

Product Dispenser

Payment System



Coil controlled by servo motor

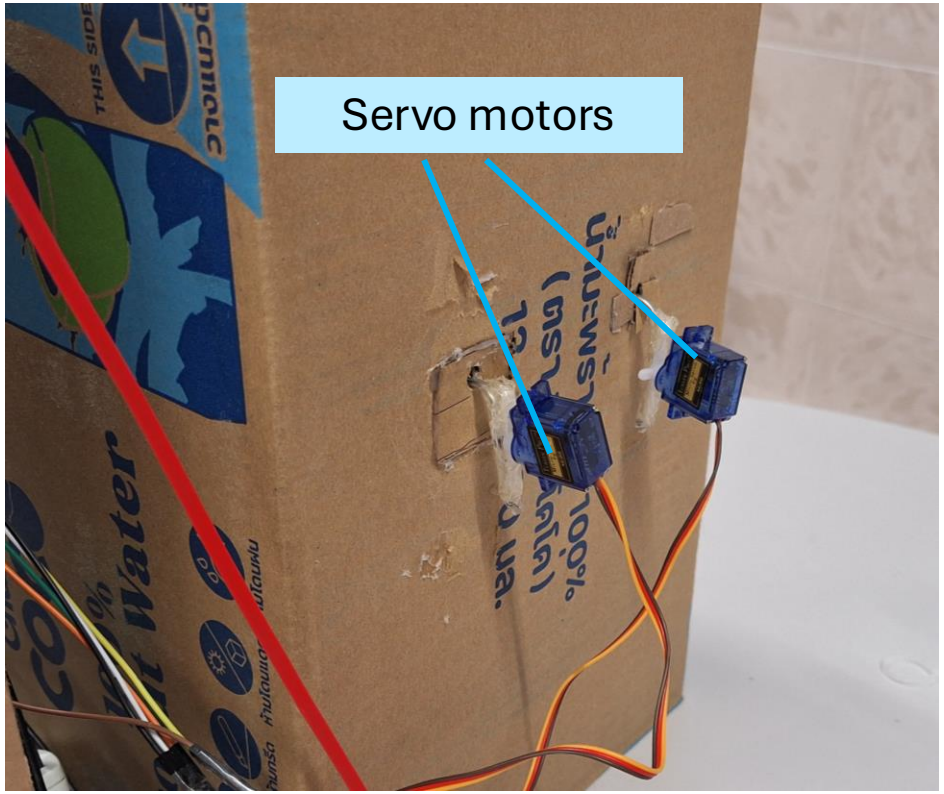
4x4 matrix keypad

RFID card reader

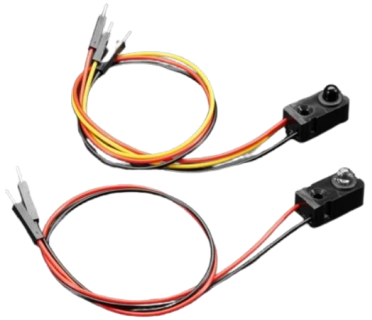
LCD display

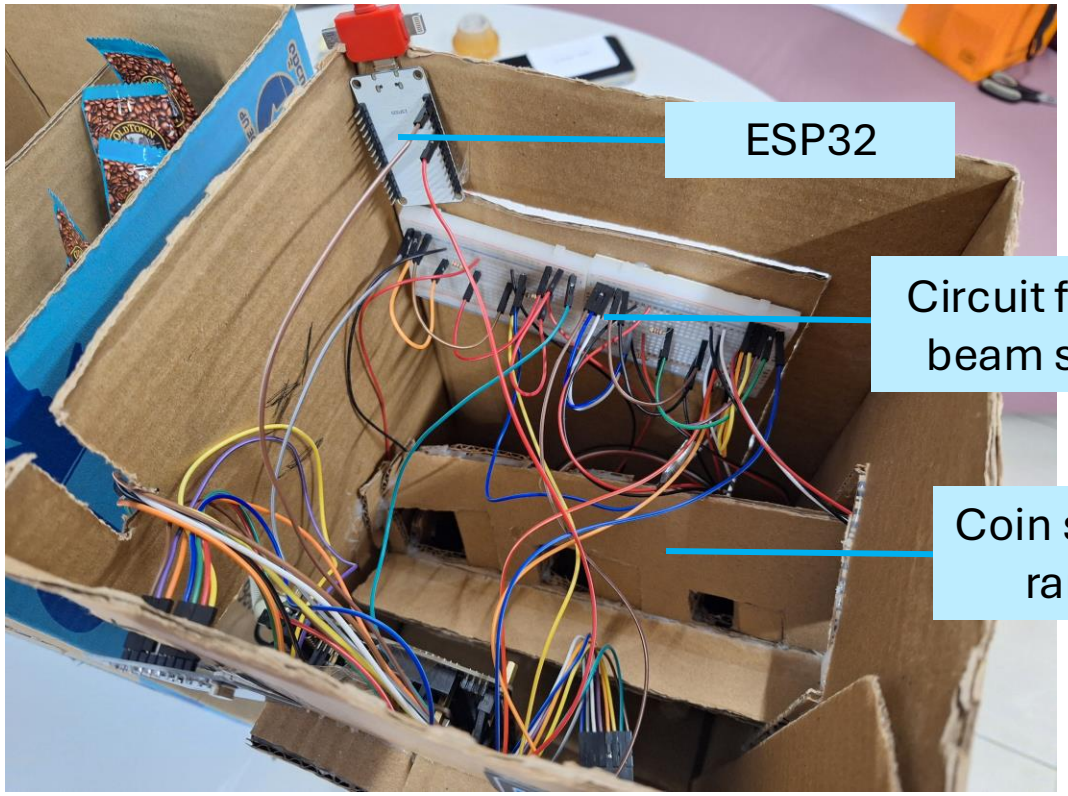
Project Design

Internal Structure of Product Dispenser



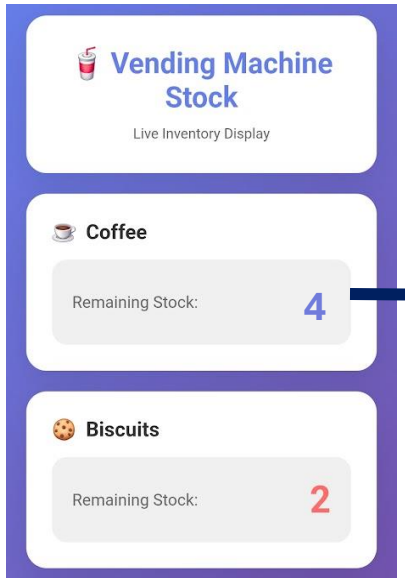
Internal Structure of Payment System





Project Workflow Diagram

Connect to SVM-WiFi
http://192.168.4.1



Select item

Select payment method

1 **Card** payment

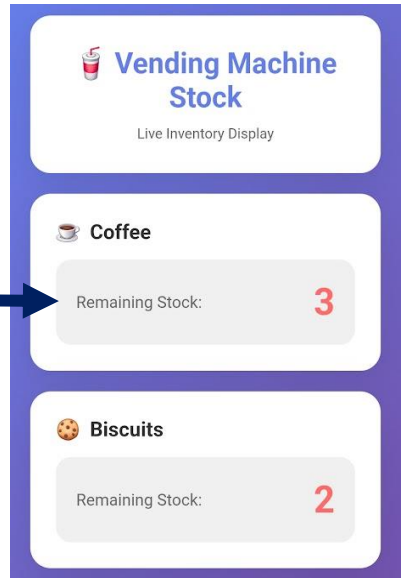
Processed by RFID reader

2 **Coin** payment

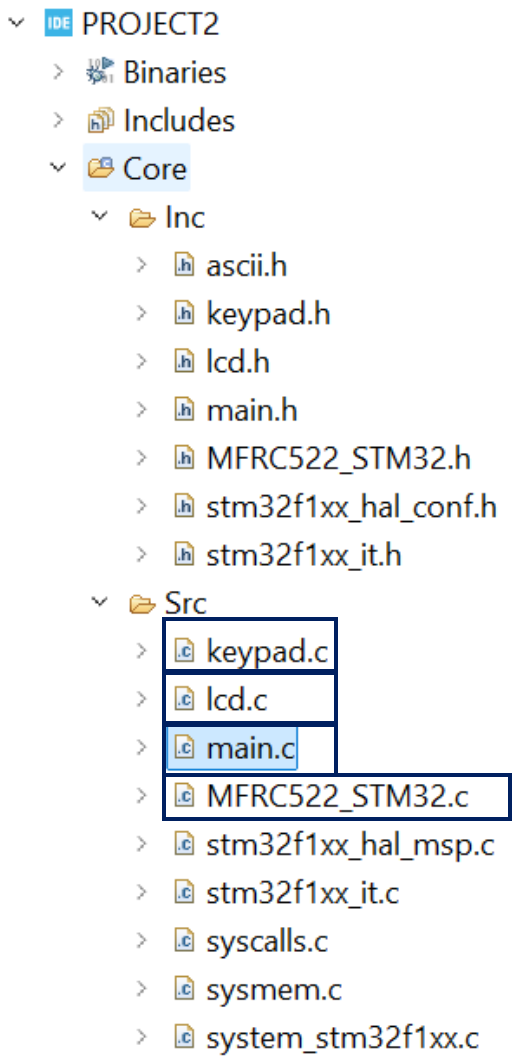
Processed by coin sorting ramp

Dispense the product by coil

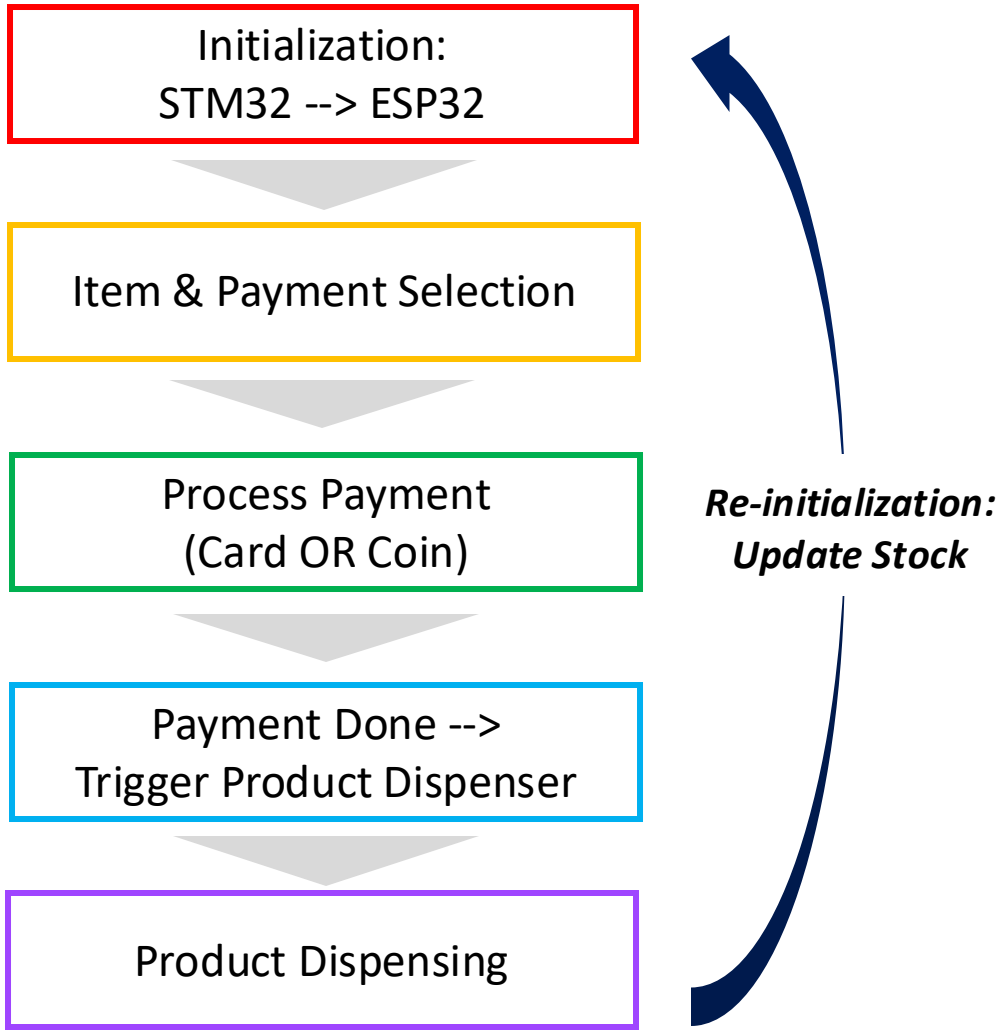
Auto-update stock



Project Workflow in Code




```c
typedef enum {
    STATE_INIT,
    STATE_SELECTION,
    STATE_PAYMENT,
    STATE_WAITING_CARD,
    STATE_WAITING_COINS,
    STATE_PAYMENT_DONE,
    STATE_DISPENSING_PRODUCT,
} Vending_Machine_State;
```



Initialization: STM32 --> ESP32 UART Communication

_STATE_INIT: *ESP32*

UART	Pins	Purpose
Transmitter	PC10	Send the stock data to the ESP32
Receiver	GPIO16 (RX2)	Receive the stock data from the STM32 and display it on the webpage

Transmitter:
- The format of the tx_buffer (transmission buffer) is "STOCK,<coffee>,<biscuits>\n"
- We send the string byte by byte

Receiver:
- Receiver receives one byte at a time until the newline character, then process the data
- Tokens are separated by a comma

*We use **UART** over SPI because:*
- Simplex 1-wire (TX/RX) setup is sufficient without SPI's extra CS pin and clock timing management
- Requires no master/slave arbitration or clock synchronization

ESP32 Wi-Fi Protocol:
- ESP32 acts as a wifi access point
  - const char* ssid = "SVM_WiFi";
  - const char* password = "12345678";

*We use **WiFi** over Bluetooth because:*
- Achieves longer range (50-70m) than bluetooth (10m)
- Supports multiple simultaneous connections

Item and Payment Selection

_STATE_SELECTION: *4x4 Keypad Matrix, LCD Display*

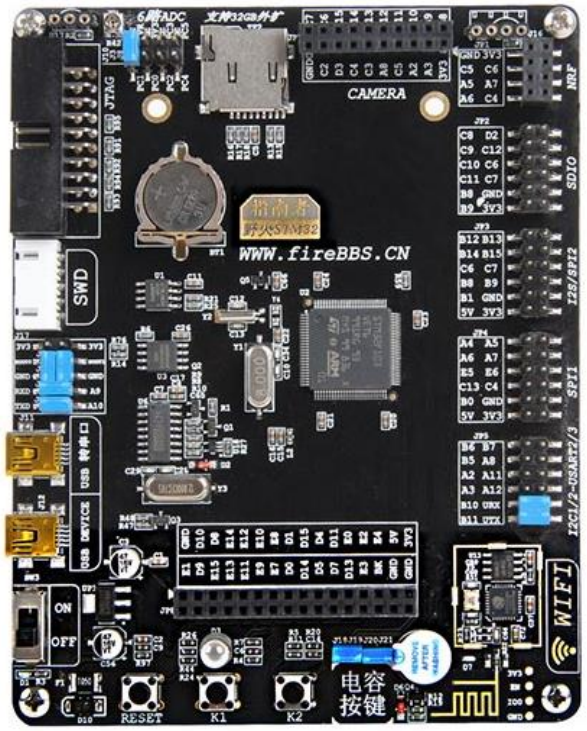
Scenario Example: Select item 1 using keypad

Input



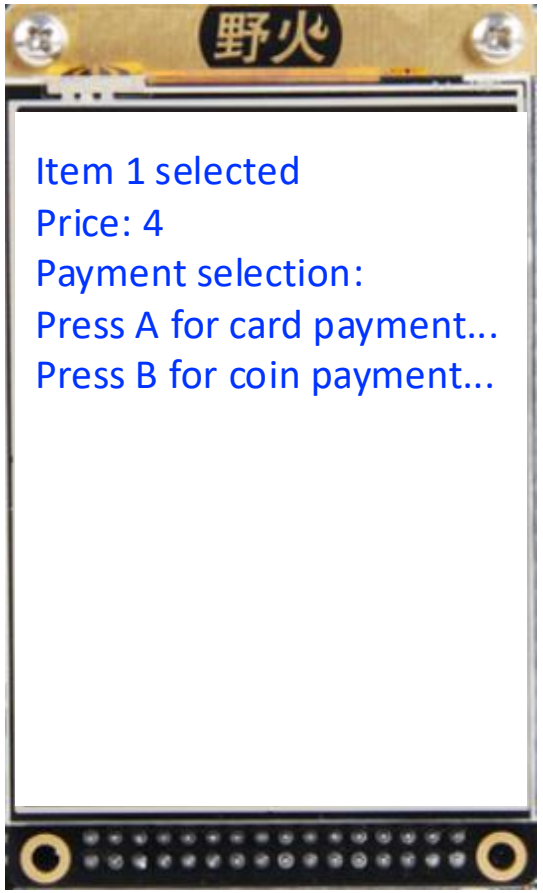
GPIO

Processor



FSMC

Output



Item and Payment Selection

_STATE_SELECTION: *4x4 Keypad Matrix + LCD Display*

		C1	C2	C3	C4
		PB6	PB5	PA2	PA3
R1	PB7	1	2	3	A
R2	PB10	4	5	6	B
R3	PA11	7	8	9	C
R4	PA12	*	0	#	D

Key Scanning Algorithm:

- Preparation:
  - Configure all rows as HIGH
  - Set all the columns to the input with pull-up
- User presses “1”
  - Row 1 and column 1 becomes connected → both become HIGH
  - Set each row to LOW one by one
    - Row 1 pulls Column 1 to LOW
    - The rest of the columns are still HIGH

Item and Payment Selection

_STATE_PAYMENT: *4x4 Keypad Matrix, LCD Display*

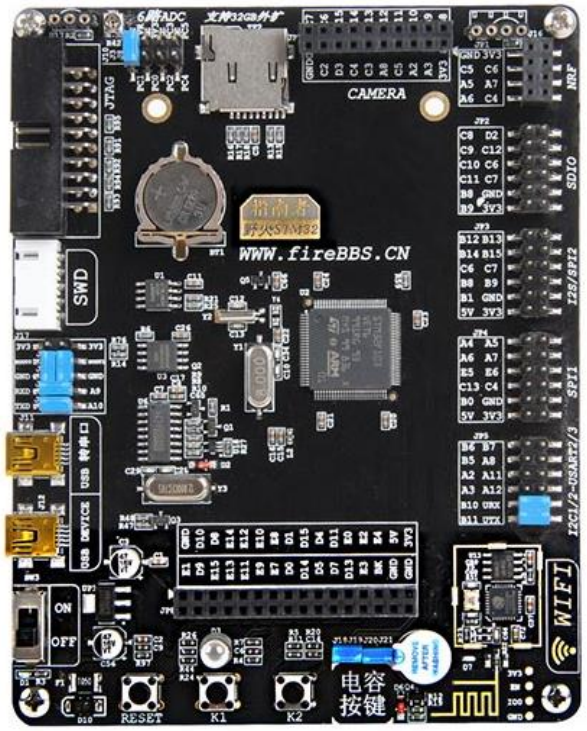
Scenario Example: Select card payment using keypad

Input



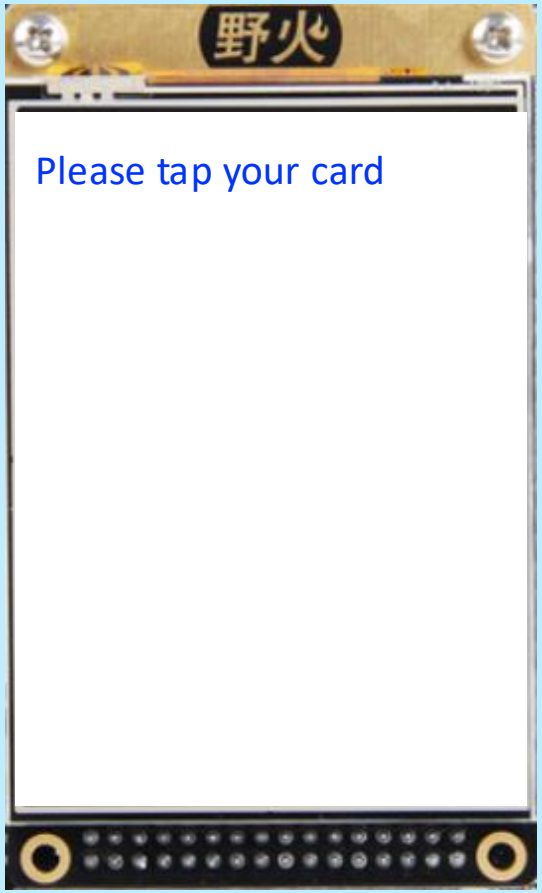
GPIO

Processor



FSMC

Output



Item and Payment Selection

_STATE_PAYMENT: *4x4 Keypad Matrix, LCD Display*

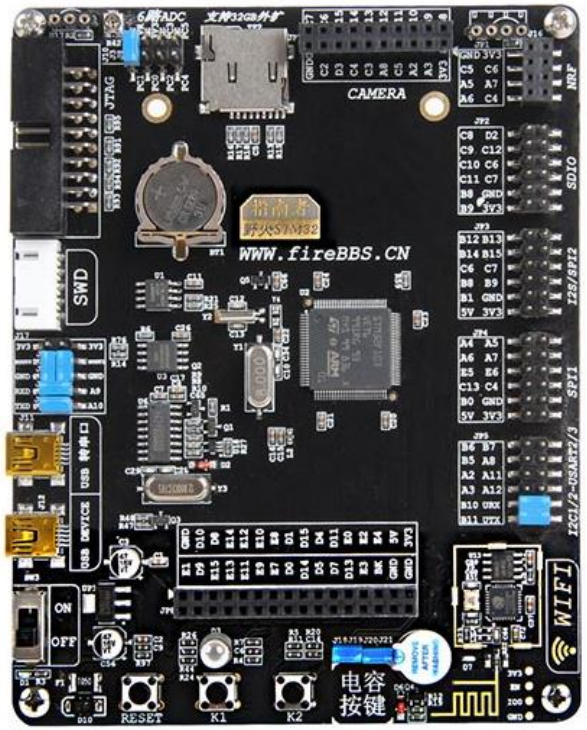
Scenario Example: Select coin payment using keypad

Input



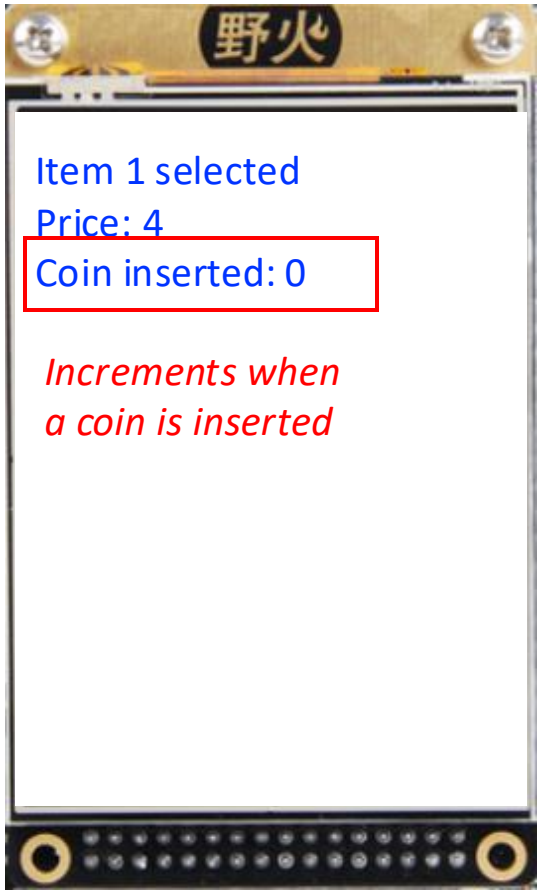
GPIO

Processor



FSMC

Output



Process Payment (Card)

_STATE_WAITING_CARD: *RFID Reader*

Name	Pins	Purpose
3.3V	3.3V	
GND	GND	
RST	PC0	Resets the RFID chip
MOSI	PB15	Master Out, Slave In (STM32 → RFID)
MISO	PB14	Master In, Slave Out (RFID → STM32)
SCK	PB13	Clock Signal, generated to synchronize data transmission on the MOSI/MISO lines
SDA	PB12	Slave Select, pulling it low enables the RFID module for SPI communication

Methodology:

1) Continuous Polling
   - STM32 repeatedly calls waitCardDetect() while in STATE_WAITING_CARD
2) Card Request
   - The reader transmits REQA (0x26) whose radio waves induce current in the card's antenna to wake it
3) Card Response
   - The card returns a 2-byte ATQA (0x42) identifying it as a "Mifare Classic" type of card
4) Anti-Collision
   - The reader picks one card, gets its unique ID, and ignores others until communication with the first card finishes.
5) UID Read & Checksum
   - Selected card transmits its unique serial number (UID) plus a checksum byte (BCC); STM32 reads one byte at a time from the reader's FIFO buffer over the SPI bus
6) Validation
   - The reader verifies integrity by comparing the received BCC against a recalculated XOR of the 4 UID bytes, returning STATUS_OK if they match

Process Payment (Card)

_STATE_WAITING_CARD: *RFID Reader*

Scenario Example: Paid with card

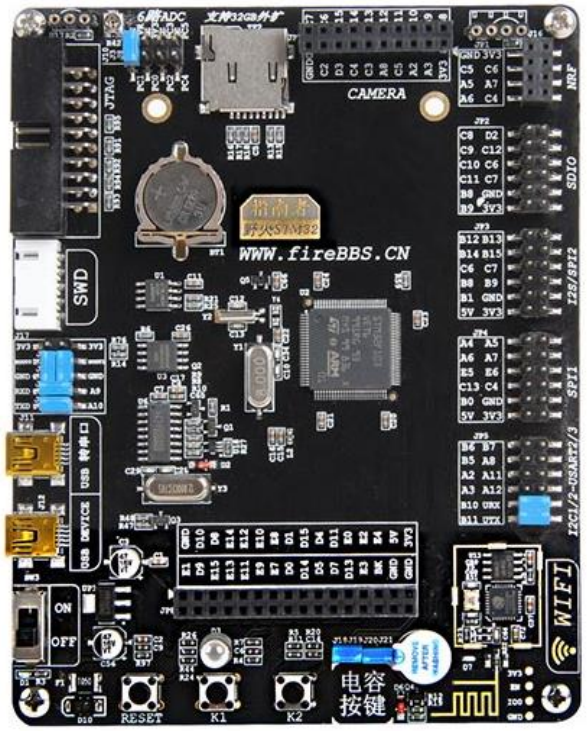
Input



MOSI

MISO

Processor



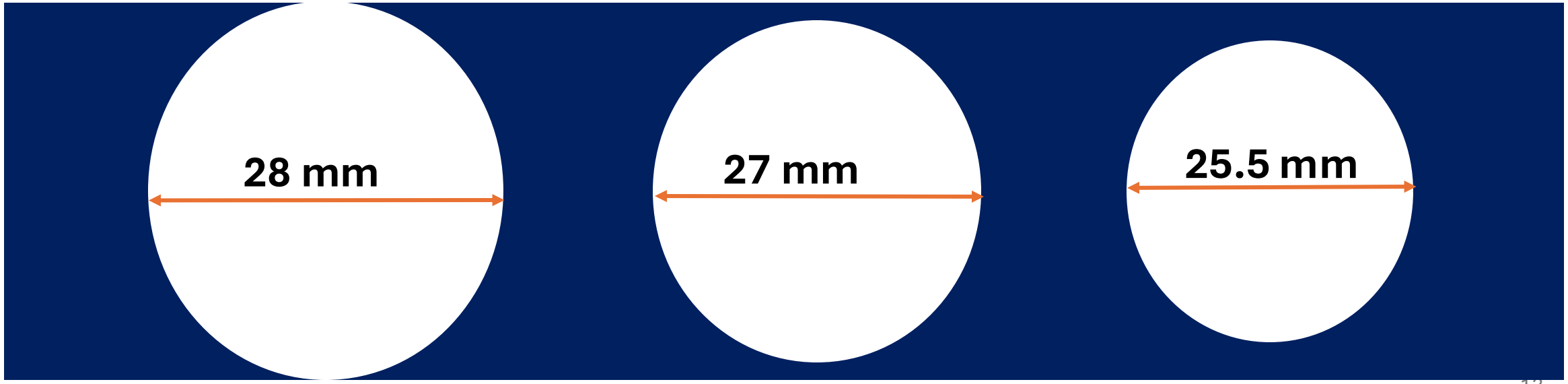
FSMC

Output



Process Payment (Coin)





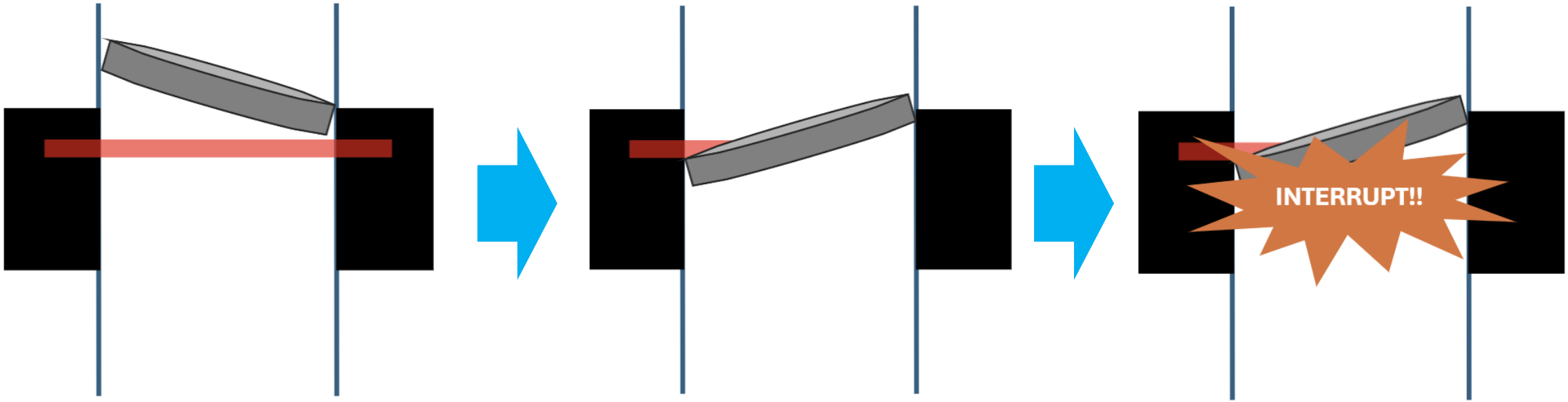
Process Payment (Coin)

_STATE_WAITING_COINS: *Break Beam Sensor, Interrupt, Ramp Mechanism*

Coins	Pins
$1	PA4
$2	PA5
$5	PA6

*We use **interrupt** over polling because:*

1) Trigger instantly when the beam breaks, while polling could miss the event due to loop delays
2) Provide consistent timing for the 20ms/50ms debounce filters, preventing double-counts without jitter caused by variable polling loop speeds.

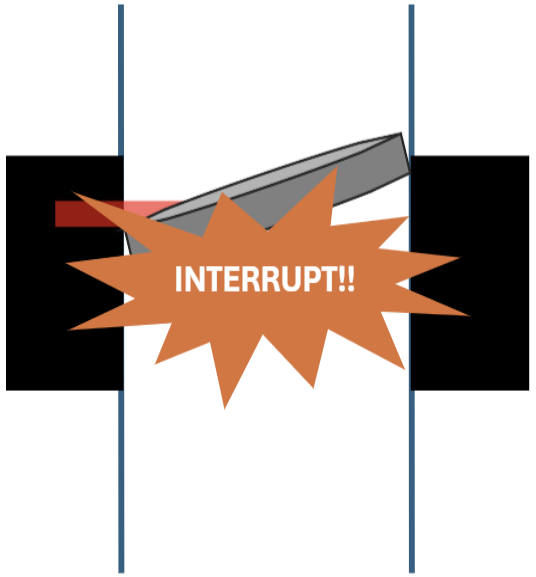


Process Payment (Card)

_STATE_WAITING_CARD: *RFID Reader*

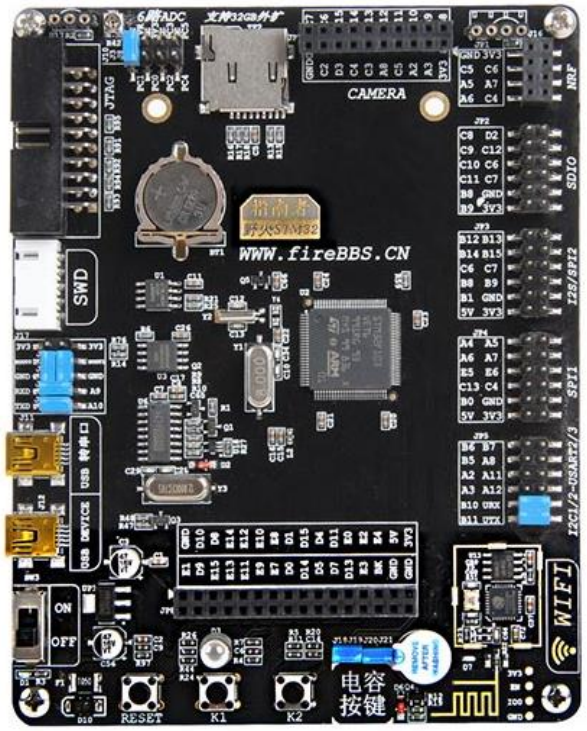
Scenario Example: Paid with card

Input



GPIO & Interrupt

Processor



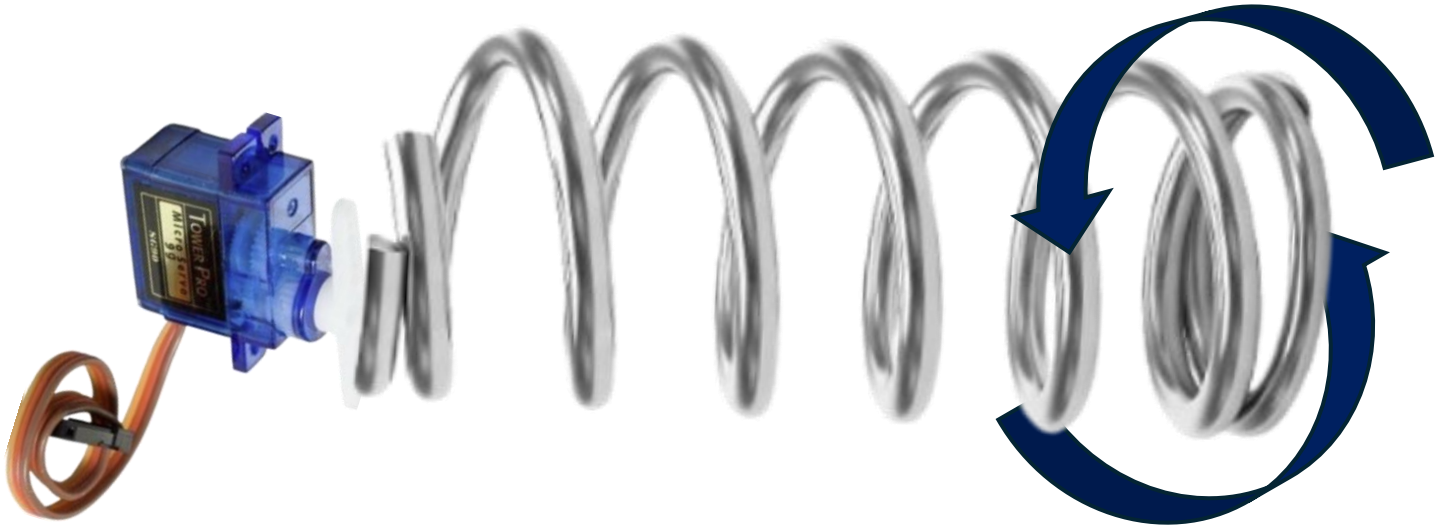
FSMC

Output



Product Dispenser

_STATE_DISPENSING_PRODUCT: *PWM Control using Timer, Continuous Servos*



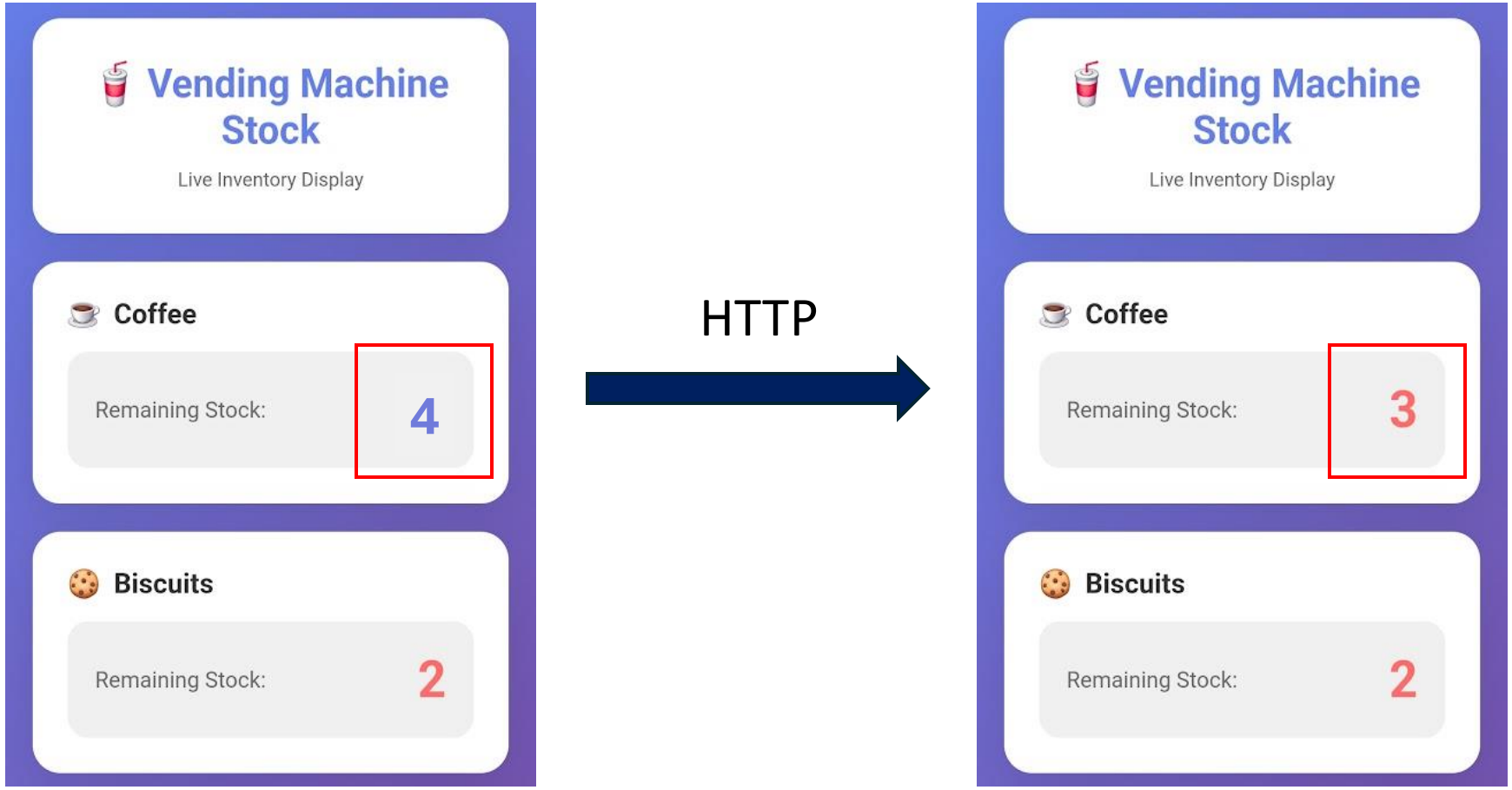
Servo	Pins	CCR
Item 1 servo	PA9	1300 for 1500ms
Item 2 servo	PB11	1100 for 1700ms

*We use **continuous servo motor** over DC motor because:*

1) Allow direct speed regulation via PWM duty cycle without complex PID tuning or external encoders to achieve slow, controlled rotation
2) Maintain commanded speed without overshoot, or risking jamming or product damage.

Re-initialization: STM32 --> ESP32 UART Communication

_STATE_INIT: *Real-Time Stock Update*



Thank you



Group 1

Sawini, Samantha